

Robot Information Publishing

Robot Information Publishing

1. Course Content
 - Course Objectives
2. Preparation
3. Demonstration
 - 3.1 Chassis Sensor Topics
 - 3.2 Controlling the Robot Chassis through Topics
 - 3.3 Controlling the Buzzer On/Off through Topics
 - 3.4 Controlling the Servo Gimbal Angle through Topics
 - 3.5 Controlling Movement through Services
 - 3.6 Saving Camera Images through a Service

1. Course Content

- Connect to the robot car through the micro-ROS agent, view topic data, and control the robot through topic and service interfaces

Course Objectives

- Understand how the car's sensor data enters the ROS 2 system
- Master the basic operations of subscribing to topics and publishing control commands through the command line

2. Preparation

```
ros2 launch microros_v2_bringup car_base.launch.py
```

```
[INFO] [ekf_node-6]: process started with pid [439721]
[micro_ros_agent-1] [1784966314.293627] info      | UDPv4AgentLinux.cpp | init                | running...         | port: 8090
[base_driver-5] [INFO] [1784966314.586459567] [base_driver]: === BaseDriverNode Parameters ===
[base_driver-5] [INFO] [1784966314.586757286] [base_driver]:   odom_linear_speed      : 0.15 m/s
[base_driver-5] [INFO] [1784966314.586950165] [base_driver]:   odom_angular_speed    : 0.5 rad/s
[base_driver-5] [INFO] [1784966314.587109546] [base_driver]:   odom_position_tolerance : 0.02 m
[base_driver-5] [INFO] [1784966314.587253347] [base_driver]:   odom_angle_tolerance  : 0.05 rad
[base_driver-5] [INFO] [1784966314.587395368] [base_driver]:   odom_timeout          : 30.0 s
[base_driver-5] [INFO] [1784966314.587544324] [base_driver]:   min_linear_speed      : 0.03 m/s
[base_driver-5] [INFO] [1784966314.587688353] [base_driver]:   min_angular_speed     : 0.2 rad/s
[base_driver-5] [INFO] [1784966314.587847813] [base_driver]:   precise_adjust_xy_pid  : [0.5, 0.0, 0.05]
[base_driver-5] [INFO] [1784966314.588051111] [base_driver]:   precise_adjust_theta_pid: [1.0, 0.0, 0.3]
[base_driver-5] [INFO] [1784966314.588210001] [base_driver]:   precise_adjust_max_lln : 0.12 m/s
[base_driver-5] [INFO] [1784966314.588352934] [base_driver]:   precise_adjust_max_ang  : 0.5 rad/s
[base_driver-5] [INFO] [1784966314.588490535] [base_driver]:   precise_adjust_pos_tol  : 0.01 m
[base_driver-5] [INFO] [1784966314.588633192] [base_driver]:   precise_adjust_ang_tol  : 0.06 rad
[base_driver-5] [INFO] [1784966314.588776923] [base_driver]: =====
[camera_driver-2] [INFO] [1784966314.832103723] [esp32_ip_camera_publisher]: Loaded calibration: 640x480, fx=1000.35, fy=999.92
[camera_driver-2] [INFO] [1784966314.832434334] [esp32_ip_camera_publisher]: CameraInfo publisher started, ost_file: /home/yahboom/microros_v2_bringup/config/ost.yaml
[camera_driver-2] [INFO] [1784966314.839272796] [esp32_ip_camera_publisher]: camera node started, camera_url: http://192.168.12.32:81/stream
[camera_driver-2] [WARN] [1784966317.842067607] [esp32_ip_camera_publisher]: Camera not opened, trying reconnect... (next retry in 1.0s)
[camera_driver-2] [INFO] [1784966318.036785177] [esp32_ip_camera_publisher]: IP camera stream opened successfully
```

3. Demonstration

3.1 Chassis Sensor Topics

- Topic summary

Topic	Message Type	Description
<code>imu</code>	<code>sensor_msgs/Imu</code>	IMU angular velocity + linear acceleration
<code>odom_raw</code>	<code>nav_msgs/Odometry</code>	Accumulated wheel odometry
<code>scan</code>	<code>sensor_msgs/LaserScan</code>	LiDAR scan data
<code>battery</code>	<code>std_msgs/Float32</code>	Battery voltage
<code>ptz_angle</code>	<code>geometry_msgs/Vector3</code>	Current gimbal angles (x=S1 horizontal, y=S2 pitch, in degrees)

- View all topics

```
ros2 topic list
```

```
yahboom@yahboom: ~ 100x15
yahboom@yahboom:~$ ros2 topic list
/battery
/beep
/camera/image_raw
/camera/image_raw/compressed
/camera/image_raw/display
/cmd_ptz_angle
/cmd_vel
/diagnostics
/imu
/imu_display
/joint_states
/odom
/odom_display
/odom_raw
/parameter_events
```

- View the publishing rate of a specified topic:

```
ros2 topic hz /imu
```

```
yahboom@yahboom: ~ 100x15
yahboom@yahboom:~$ ros2 topic hz /imu_display
average rate: 27.648
  min: 0.014s max: 0.055s std dev: 0.00891s window: 29
average rate: 27.612
  min: 0.004s max: 0.056s std dev: 0.00915s window: 57
average rate: 27.187
  min: 0.004s max: 0.069s std dev: 0.00860s window: 85
average rate: 27.582
  min: 0.004s max: 0.069s std dev: 0.00868s window: 114
average rate: 27.653
  min: 0.004s max: 0.069s std dev: 0.00801s window: 142
average rate: 27.657
  min: 0.004s max: 0.069s std dev: 0.00762s window: 170
average rate: 27.661
  min: 0.004s max: 0.069s std dev: 0.00769s window: 198
average rate: 27.479
```

- View the data content of a specified topic

```
ros2 topic echo /imu --once
```

```
yahboom@yahboom:~$ ros2 topic echo /imu --once
header:
  stamp:
    sec: 1784971112
    nanosec: 194000000
  frame_id: imu_frame
orientation:
  x: -0.003829518100246787
  y: -0.0006782016716897488
  z: 2.0247578504495323e-05
  w: 1.0001169443130493
orientation_covariance:
  - 0.01
  - 0.0
  - 0.0
  - 0.0
  - 0.01
  - 0.0
  - 0.0
  - 0.0
  - 0.01
angular_velocity:
  x: 0.0023198332637548447
```

[!TIP]

The command format for viewing the data content, rate, and other information of other topics is the same; just replace the corresponding topic name

3.2 Controlling the Robot Chassis through Topics

- Publish velocity commands to `/cmd_vel` to control the car movement:

```
# Move forward at 0.3 m/s on the x-axis
ros2 topic pub /cmd_vel geometry_msgs/msg/Twist "{linear: {x: 0.3}, angular: {z: 0.0}}"
```

```
yahboom@yahboom: ~ 107x17
yahboom@yahboom:~$ ros2 topic pub /cmd_vel geometry_msgs/msg/Twist "{linear: {x: 0.3}, angular: {z: 0.0}}"
publisher: beginning loop
publishing #1: geometry_msgs.msg.Twist(linear=geometry_msgs.msg.Vector3(x=0.3, y=0.0, z=0.0), angular=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=0.0))

publishing #2: geometry_msgs.msg.Twist(linear=geometry_msgs.msg.Vector3(x=0.3, y=0.0, z=0.0), angular=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=0.0))

publishing #3: geometry_msgs.msg.Twist(linear=geometry_msgs.msg.Vector3(x=0.3, y=0.0, z=0.0), angular=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=0.0))

publishing #4: geometry_msgs.msg.Twist(linear=geometry_msgs.msg.Vector3(x=0.3, y=0.0, z=0.0), angular=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=0.0))

publishing #5: geometry_msgs.msg.Twist(linear=geometry_msgs.msg.Vector3(x=0.3, y=0.0, z=0.0), angular=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=0.0))
```

Turn left in place at 1.0 rad/s

```
ros2 topic pub /cmd_vel geometry_msgs/msg/Twist "{angular: {z: 1.0}}"
```

```
yahboom@yahboom: ~ 111x24
yahboom@yahboom:~$ ros2 topic pub /cmd_vel geometry_msgs/msg/Twist "{linear: {x: 0.0}, angular: {z: 1.0}}"
publisher: beginning loop
publishing #1: geometry_msgs.msg.Twist(linear=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=0.0), angular=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=1.0))

publishing #2: geometry_msgs.msg.Twist(linear=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=0.0), angular=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=1.0))

publishing #3: geometry_msgs.msg.Twist(linear=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=0.0), angular=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=1.0))

publishing #4: geometry_msgs.msg.Twist(linear=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=0.0), angular=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=1.0))

publishing #5: geometry_msgs.msg.Twist(linear=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=0.0), angular=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=1.0))

publishing #6: geometry_msgs.msg.Twist(linear=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=0.0), angular=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=1.0))

publishing #7: geometry_msgs.msg.Twist(linear=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=0.0), angular=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=1.0))

publishing #8: geometry_msgs.msg.Twist(linear=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=0.0), angular=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=1.0))
```

- Stop the car movement

```
ros2 topic pub /cmd_vel geometry_msgs/msg/Twist "{linear: {x: 0.0}, angular: {z: 0.0}}"
```

3.3 Controlling the Buzzer On/Off through Topics

```
# Turn on the buzzer
ros2 topic pub /beep std_msgs/msg/UInt16 "{data: 1}" --once
# Turn off the buzzer
ros2 topic pub /beep std_msgs/msg/UInt16 "{data: 0}" --once
```

```

yahboom@yahboom:~$ ros2 topic pub /beep std_msgs/msg/UInt16 "{data: 1}" --once
publisher: beginning loop
publishing #1: std_msgs.msg.UInt16(data=1)

yahboom@yahboom:~$ ros2 topic pub /beep std_msgs/msg/UInt16 "{data: 0}" --once
publisher: beginning loop
publishing #1: std_msgs.msg.UInt16(data=0)

```

3.4 Controlling the Servo Gimbal Angle through Topics

[!NOTE]

x:0.0 ,y: -45.0 is the neutral position of the servo gimbal: the camera faces directly forward

```

# Control the servo gimbal to the specified angle
ros2 topic pub /cmd_ptz_angle geometry_msgs/msg/Vector3 "{x: 10.0, y: 15.0}" --once
# Return the servo gimbal to the neutral position
ros2 topic pub /cmd_ptz_angle geometry_msgs/msg/Vector3 "{x: 0.0, y: -45.0}" --once

```

3.5 Controlling Movement through Services

- Use the action service to move or rotate the chassis by a precise distance (based on odometry closed-loop control)

[!NOTE]

Parameter description:

direction: movement direction; distance: movement distance; angle: rotation angle

- Usage examples

```

# Move forward 0.15 meters
ros2 action send_goal /move_by_odom interfaces/action/MoveByOdom "{direction: 'forward', distance: 0.15, angle: 0.0}"
# Move backward 0.3 meters
ros2 action send_goal /move_by_odom interfaces/action/MoveByOdom "{direction: 'backward', distance: 0.3, angle: 0.0}"

```

```

yahboom@yahboom:~$ ros2 action send_goal /move_by_odom interfaces/action/MoveByOdom "{direction: 'forward', distance: 0.15, angle: 0.0}"
Waiting for an action server to become available...
Sending goal:
  distance: 0.15
direction: forward
angle: 0.0

Goal accepted with ID: 42c79e994a41410cb5f2a1f95add6886

Result:
  success: true
message: Moved forward 0.130m

Goal finished with status: SUCCEEDED

```

```
# Turn left 90 degrees
ros2 action send_goal /move_by_odom interfaces/action/MoveByOdom "{direction:
'turn_left', distance: 0.0, angle: 90.0}"
# Turn right 45 degrees
ros2 action send_goal /move_by_odom interfaces/action/MoveByOdom "{direction:
'turn_right', distance: 0.0, angle: 45.0}"
```

```
yahboom@yahboom:~$ ros2 action send_goal /move_by_odom interfaces/action/MoveByOdom "{direction: 'turn_left', distance: 0.0, angle: 90.0}"
Waiting for an action server to become available...
Sending goal:
  distance: 0.0
direction: turn_left
angle: 90.0

Goal accepted with ID: ad804653e7084e20a65b97a78476d2be

Result:
  success: true
message: Rotated turn_left 90.0°

Goal finished with status: SUCCEEDED
```

3.6 Saving Camera Images through a Service

- Save the latest image from the camera's video stream

```
ros2 service call /save_image std_srvs/srv/Trigger
```

```
yahboom@yahboom:~$ ros2 service call /save_image std_srvs/srv/Trigger
waiting for service to become available...
requester: making request: std_srvs.srv.Trigger_Request()

response:
std_srvs.srv.Trigger_Response(success=True, message='/home/yahboom/microros_ws/cache/image.png')
```

- Default image save path: `$HOME/microros_ws/cache/image.png`. You can view it at the corresponding path in the virtual machine

```
yahboom@yahboom:~$ ls $HOME/microros_ws/cache/image.png
/home/yahboom/microros_ws/cache/image.png
```